

1. Employee/Student Problem

A. Problem Statement

Given a list of employees (or students) with their IDs and marks/salary, find a specific record efficiently and perform operations like sorting or searching.

B. Strategy

We use basic searching and sorting techniques. For efficient search, Binary Search is used after sorting the data.

C. Algorithm

1. Input number of records
2. Store data in array
3. Sort array based on ID
4. Apply Binary Search to find required record

D. Source Code (C++)

```
#include <iostream>
using namespace std;
struct Student {
    int id;
    int marks;
};

int binarySearch(Student arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid].id == key)
            return mid;
        else if (arr[mid].id < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main() {
    int n;
    cin >> n;
```

```
Student arr[n];

for(int i = 0; i < n; i++)
    cin >> arr[i].id >> arr[i].marks;

int key;
cin >> key;

int result = binarySearch(arr, n, key);

if(result != -1)
    cout << "Found: " << arr[result].marks;
else
    cout << "Not Found";

return 0;
}
```

E. Time Complexity

- Binary Search: $O(\log n)$
 - Sorting (if applied): $O(n \log n)$
-

2. Divide and Conquer

A. Problem Statement

Sort an array using Divide and Conquer approach.

B. Strategy

Divide the array into two halves, sort them recursively, and then merge.

C. Algorithm (Merge Sort)

1. Divide array into two halves
2. Recursively sort both halves
3. Merge sorted halves

D. Source Code (C++)

```
#include <iostream>
using namespace std;

void merge(int arr[], int l, int m, int r) {
    int i = l, j = m+1, k = 0;
    int temp[100];

    while(i <= m && j <= r) {
        if(arr[i] < arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while(i <= m)
        temp[k++] = arr[i++];
    while(j <= r)
        temp[k++] = arr[j++];

    for(int i = l; i <= r; i++)
        arr[i] = temp[i-l];
}

void mergeSort(int arr[], int l, int r) {
    if(l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m+1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int n;
    cin >> n;
    int arr[n];

    for(int i = 0; i < n; i++)
        cin >> arr[i];

    mergeSort(arr, 0, n-1);

    for(int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

}

E. Time Complexity

- Best, Average, Worst: $O(n \log n)$
-

3. Dynamic Programming

A. Problem Statement

Find nth Fibonacci number using Dynamic Programming.

B. Strategy

Store previously computed results to avoid recomputation (Tabulation).

C. Algorithm

1. Initialize array
2. Store base cases
3. Fill remaining values using formula

D. Source Code (C++)

```
#include <iostream>
using namespace std;

int fib(int n) {
    int dp[n+1];
    dp[0] = 0;
    dp[1] = 1;

    for(int i = 2; i <= n; i++)
        dp[i] = dp[i-1] + dp[i-2];

    return dp[n];
}

int main() {
    int n;
    cin >> n;
    cout << fib(n);
    return 0;
}
```

E. Time Complexity

- $O(n)$
-

4. Greedy Algorithm

A. Problem Statement

Select maximum number of activities that do not overlap.

B. Strategy

Always pick the activity that finishes earliest.

C. Algorithm

1. Sort activities by finish time
2. Select first activity
3. Select next activity whose start $\geq$ last finish

D. Source Code (C++)

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Activity {
    int start, finish;
};

bool compare(Activity a, Activity b) {
    return a.finish < b.finish;
}

int main() {
    int n;
    cin >> n;
    Activity arr[n];

    for(int i = 0; i < n; i++)
        cin >> arr[i].start >> arr[i].finish;

    sort(arr, arr+n, compare);

    int count = 1;
    int lastFinish = arr[0].finish;

    for(int i = 1; i < n; i++) {
```

```

        if(arr[i].start >= lastFinish) {
            count++;
            lastFinish = arr[i].finish;
        }
    }

    cout << count;
    return 0;
}

```

E. Time Complexity

- Sorting: $O(n \log n)$
 - Selection: $O(n)$
-

5. Backtracking

A. Problem Statement

Solve N-Queens problem.

B. Strategy

Place queens one by one and backtrack if conflict occurs.

C. Algorithm

1. Place queen in row
2. Check if safe
3. Recur for next row
4. Backtrack if needed

D. Source Code (C++)

```

#include <iostream>
using namespace std;

#define N 4

bool isSafe(int board[N][N], int row, int col) {
    for(int i = 0; i < col; i++)
        if(board[row][i])
            return false;

    for(int i=row, j=col; i>=0 && j>=0; i--, j--)
        if(board[i][j])

```

```

        return false;

    for(int i=row, j=col; i<N && j>=0; i++, j--)
        if(board[i][j])
            return false;

    return true;
}

bool solve(int board[N][N], int col) {
    if(col >= N)
        return true;

    for(int i = 0; i < N; i++) {
        if(isSafe(board, i, col)) {
            board[i][col] = 1;
            if(solve(board, col+1))
                return true;
            board[i][col] = 0;
        }
    }
    return false;
}

int main() {
    int board[N][N] = {0};

    if(solve(board, 0)) {
        for(int i=0; i<N; i++) {
            for(int j=0; j<N; j++)
                cout << board[i][j] << " ";
            cout << endl;
        }
    } else {
        cout << "No solution";
    }

    return 0;
}

```

E. Time Complexity

- $O(N!)$ (worst case)
-

6. Branch and Bound

A. Problem Statement

Solve 0/1 Knapsack using Branch and Bound.

B. Strategy

Explore solution space using bounds to prune unnecessary branches.

C. Algorithm

1. Sort items by value/weight ratio
2. Use queue to explore nodes
3. Compute bound and prune

D. Source Code (C++)

```
#include <iostream>
#include <queue>
using namespace std;

struct Item {
    int weight, value;
};

struct Node {
    int level, profit, weight;
    float bound;
};

float bound(Node u, int n, int W, Item arr[]) {
    if(u.weight >= W)
        return 0;

    float profit_bound = u.profit;
    int j = u.level + 1;
    int totweight = u.weight;

    while(j < n && totweight + arr[j].weight <= W) {
        totweight += arr[j].weight;
        profit_bound += arr[j].value;
        j++;
    }

    if(j < n)
        profit_bound += (W - totweight) * ((float)arr[j].value / arr[j].weight);

    return profit_bound;
}
```

```

int knapsack(int W, Item arr[], int n) {
    queue<Node> Q;
    Node u, v;

    v.level = -1; v.profit = 0; v.weight = 0;
    Q.push(v);

    int maxProfit = 0;

    while(!Q.empty()) {
        v = Q.front(); Q.pop();

        if(v.level == n-1)
            continue;

        u.level = v.level + 1;

        u.weight = v.weight + arr[u.level].weight;
        u.profit = v.profit + arr[u.level].value;

        if(u.weight <= W && u.profit > maxProfit)
            maxProfit = u.profit;

        u.bound = bound(u, n, W, arr);

        if(u.bound > maxProfit)
            Q.push(u);

        u.weight = v.weight;
        u.profit = v.profit;
        u.bound = bound(u, n, W, arr);

        if(u.bound > maxProfit)
            Q.push(u);
    }

    return maxProfit;
}

int main() {
    int n, W;
    cin >> n >> W;

    Item arr[n];
    for(int i = 0; i < n; i++)
        cin >> arr[i].weight >> arr[i].value;
}

```

```
cout << knapsack(W, arr, n);  
    return 0;  
}
```

E. Time Complexity

- Worst case: **Exponential**
- Better than brute force due to pruning